GAME DESIGN RESEARCH / NOVEMBER 2026

Free Games on GitHub

What makes them engaging.
How they are built.
What Claude should build next.

13 source-based case studies
6 original game briefs
A concrete implementation handoff

Recommended first project: One More Season
Broader-audience alternative: Blackout Dispatch

Prepared for Kevin

November 5, 2026

Contents

How to use this report . . . 4

Research scope and evidence limits . . . 4

Selected repositories, ordered by observed stars . . . 5

What was screened but not made a main case . . . 6

Game case studies **7**

1. Mindustry - make the system's weaknesses visible . . . 7

2. osu!lazer - repeatable mastery supported by a creator ecosystem . . . 8

3. Lichess: make improvement part of playing . . . 9

4. OpenRA - opportunity costs across economy, territory, and combat . . . 10

5. Cataclysm: Dark Days Ahead - stories emerge from interacting constraints . . . 11

6. 2048: a small ruleset with persistent spatial consequences . . . 13

7. Unciv - connect the next turn to a long-term identity . . . 14

8. OpenTTD - let players program a useful network . . . 15

9. A Dark Room: progression changes the kind of attention required . . . 16

10. Endless Sky - make growth change both freedom and responsibility . . . 18

11. Shattered Pixel Dungeon - constrained randomness and decisions that cost time . . . 19

12. The Evolution of Trust: make the player's prediction the hook . . . 20

13. Bitburner: automation becomes both the tool and the reward . . . 22

Supplemental design note: shapez . . . 24

What Claude should build from these lessons **25**

Recommendation: build one small, complete game first . . . 25

The design principles worth carrying forward . . . 25

Engineering guidance for Claude . . . 27

Six original build briefs 28
Brief A: One More Season 28
Brief B: Blackout Dispatch 30
Brief C: The Last Signal 31
Brief D: Junkyard Courier 31
Brief E: Beatline Salvage 32
Brief F: Roommate Republic 33
A practical build sequence 34
Playtest protocol and stop rules 34
Copy/paste master instruction for Claude 34

Evidence, reuse, and next decision **36**

What is established and what remains a hypothesis 36
Reuse boundary 36
Source reading plan for a future implementation 36
The next concrete step 37

Prepared for Kevin | Research snapshot: November 5, 2026

This report studies 13 games and playable experiences with public GitHub source, explains the decision patterns that may make them engaging, examines real implementation boundaries, and translates the lessons into six original projects that Claude can build. It is intended to be usable as both a reading report and an implementation handoff.

Main finding: the most transferable feature is a repeating decision that becomes more interesting as the player learns. Attractive progression connects that decision to an observable consequence, a new possibility, or a better understanding of the system. A large feature inventory does not establish that this connection exists.

Recommended starting point for Kevin: Brief A, **One More Season**, a six-fixture sports management game. **Recommended broader-audience experiment:** Brief B, **Blackout Dispatch**, a twelve-turn network puzzle. Build one first; the briefs are alternatives, not a combined feature list.

How to use this report

1. Read the research scope and repository table below.
2. Read the case studies for the kinds of decisions you want players to enjoy.
3. Select one original brief in the design section.
4. Give Claude this Markdown report and the separate start-here handoff. The start-here file is self-contained for the recommended sports brief.
5. Require a complete playable run, verified recovery, and an observed playtest before expanding the scope.

Research scope and evidence limits

The discovery pass screened more than twenty public repositories using GitHub topic listings, an open-source game directory, repository metadata, and official project material. Thirteen cases were selected for a combination of visible community interest, a no-cost playable route, inspectable gameplay code, and different transferable design lessons. This is a curated research set, not an exhaustive global top-thirteen ranking. Large engines, game lists, asset-dependent ports, and duplicates can otherwise dominate a search for popular games.

Popularity measurement. The table uses direct GitHub repository metadata retrieved on November 5, 2026. Stars indicate repository interest; forks indicate code reuse or development activity. Neither measures unique players, daily activity, retention, sales, or enjoyment. Counts change, so exact values below are a dated snapshot. The numbers are sorted within this selected set only.

Player adoption. Official distribution routes and community features provide context. Where direct operator data exists, it is labeled: Lichess's About page says more than five million games are played daily. That counts games rather than people. Public Steam ratings or app-store installs, when mentioned, are platform-specific observations and should not be combined with stars into a single popularity score. Lichess About.

Review depth. The study inspected project documentation and targeted gameplay/source files, including implementation of loops, rules, progression, content, timing, or recovery. Source-reading maps are included in the cases. The games were not compiled and exhaustively playtested in this research session.

Consequently, the report does not certify current build health, absence of bugs, device compatibility, or measured player psychology.

Interpretation. The mechanics and code paths are observed evidence. Explanations of why they may be fun, why a distribution choice may help, and which new game should be built are design inferences. They are hypotheses to test with players. The report does not infer causality from a star count or claim that a new game will inherit an established game's audience.

Meaning of free. Inclusion means there is a documented no-cost way to play the discussed experience, sometimes alongside paid convenience editions. It does not mean every store edition is free, every third-party asset is reusable, or every project uses a permissive code license. OpenRA is an engine plus supplied game mods and external assets; Lichess is an online service; The Evolution of Trust is a finite playable explanation. These differences are stated rather than hidden.

Discovery references: GitHub game topic sorted by stars and Open Source Games List. Discovery catalogs were used to find candidates; technical claims rely on the projects' own source and documentation.

Selected repositories, ordered by observed stars

Game and repository	Stars	Forks	Main transferable lesson
Mindustry	29,673	3,864	Visible automation under pressure
osu!lazer	19,376	2,873	Precision, practice, and expressive content
Lichess / Lila	18,946	2,805	Play, review, improve, return
OpenRA	17,737	3,111	Composition, counters, and information
Cataclysm: Dark Days Ahead	13,468	4,719	Interacting systems create stories
2048	13,465	17,701	A tiny action space with consequential moves
Unciv	11,528	1,995	Competing investments and future commitments
OpenTTD	8,461	1,331	Network bottlenecks become self-directed goals
A Dark Room	8,380	1,847	Progressive discovery changes the game
Endless Sky	7,726	1,349	Jobs, loadouts, and a self-chosen career
Shattered Pixel Dungeon	6,764	1,720	Scarcity, knowledge, and changing runs
The Evolution of Trust	6,339	1,007	Prediction, experiment, and revelation
Bitburner (active repo)	1,638	511	Automation itself becomes the reward

Metadata source for every row: the corresponding GitHub REST repository endpoint, `https://api.github.com/repos/OWNER/REPOSITORY`, using the exact owner/repository in the linked project. The companion case notes explain the free route and source licensing. A lack of an automatic SPDX result is not evidence that a project has no license.

Bitburner needs special treatment. The active repository has 1,638 stars. The older danielyxie/bitburner repository had 2,846 stars and 725 forks in this snapshot, and was not marked archived; its README explicitly redirects development and documentation to the new repository. Those are separate repository counts and are not added together. Its Steam page was free to play and showed 6,060 English reviews with 94% positive at retrieval. That is a storefront-specific satisfaction signal, not an active-player count. Bitburner is included for its distinctive playable programming/automation loop, not because its active repository ranks alongside Mindustry in stars.

What was screened but not made a main case

Project	Stars	Decision
OpenRCT2	16,544	Its engine is free, but ordinary use requires RollerCoaster Tycoon 2 assets. Excluded from the main free-game set.
Luanti	13,818	A voxel game creation platform hosting many separate games. Useful technology reference, not one comparable game.
DevilutionX	9,988	Full Diablo requires original game data; free shareware covers only a portion. Excluded from the main full-game set.
shapez	7,003	Public source and an online demo; normal full-game distribution and bundled-asset terms need separate treatment. Included only as a supplemental design note.
The Battle for Wesnoth	6,976	A credible free turn-based strategy candidate; left outside this deep-dive set to limit overlapping strategy coverage.
OpenMW	6,721	Its own README requires ownership of Morrowind to play Morrowind. Engine study is a different task.
SuperTuxKart	5,466	A worthwhile racing reference; outside this initial set, which favors transferable systems and achievable small projects.
Friday Night Funkin'	3,830	A credible rhythm candidate; osu! was selected for the mastery and tooling case.
Hextris	2,443	A credible small puzzle candidate; 2048 was selected for the compact interaction case.

These are scope decisions, not negative quality judgments. Other screened repositories included Evolve, DDraceNetwork, and Candy Box 2. No claim is made that the selected set covers every genre or every popular free game.

Free-access distinctions are supported by OpenRCT2 installation requirements, OpenMW's README, DevilutionX installation instructions, Luanti's official description, and shapez's README.

Game case studies

Read these as a collection of reusable decisions and implementation techniques. Each case distinguishes observable mechanics from a proposed explanation of their appeal.

1. Mindustry - make the system's weaknesses visible

Premise and availability. Mindustry combines resource automation, tower defense, and real-time strategy. The player mines resources, builds supply chains, powers production, supplies defenses or units, survives pressure, and expands. Its official itch page offers name-your-own-price downloads, including a zero-price option; the README also lists mobile and desktop distribution channels. The Steam edition is paid and adds platform features. Calling every edition free would be inaccurate. README · Official downloads and game description

The decisions that create the game. A factory's output must reach something useful. An ammunition shortage can be a mining, processing, routing, power, or consumption problem. The player allocates scarce space and resources between expanding income, improving reliability, and defending what already exists. Preparation has an external test: enemies arrive and reveal whether the supply system actually works. This provides a short loop of observation and repair inside a longer loop of research and territorial expansion.

Design inference: why this can be fun. The strongest ingredient is a correctable problem that can be seen moving across the map. A blocked belt produces a concrete question: where is the blockage, and what change will clear it? Successful repairs produce immediate visual evidence. Defense gives throughput a purpose and creates a reason to improve beyond an aesthetically tidy factory. The player alternates between deliberate construction and a stressful test of that construction. A design borrowing this lesson should make one bottleneck understandable before adding several resource types.

How it is made. Current source uses Java with the author's Arc framework and Gradle, with desktop, server, Android, and iOS modules. The build declares Arc dependencies; older descriptions that loosely call the current implementation LibGDX should not replace the inspected build. The README describes generated networking and entity code, including `mindustry.gen`, `@Remote`, and entity components. Build

Three inspected files connect the implementation to the experience:

- `Conveyor.java` updates individual item positions, respects spacing and the receiving conveyor's available space, and removes an item after a successful transfer. An empty belt can sleep. Downstream congestion therefore propagates through local rules. Visible item movement is both presentation and a diagnostic tool.
- `Waves.java` builds enemy groups with difficulty, unit tiers, air or naval constraints, overlapping groups, bosses, and shields. Pressure varies through composition and timing, giving the player reasons to change defenses beyond increasing damage numbers.
- `Logic.java` owns wave timing and progression, invokes spawning, increments the wave, and emits events. Preparation time and the arrival of pressure have explicit rules rather than being scattered across interface actions.

Popularity and reach. Free downloads, multiple operating systems, multiplayer, user maps, schematics, modding, and accessible documentation are documented channels for adoption and continued participation. They plausibly help people encounter the game and share solutions. The star count supports substantial developer attention; it does not prove these mechanisms caused that attention or establish a player population. Official page · Wiki

What Claude should learn. Build a small simulation with explicit producer, transport, buffer, and consumer rules. Expose capacity, blockage, and output. Let players make a repair and see the next test improve. Start with one resource and one pressure source. For a management game, the equivalent may be staffing, equipment, or a training queue; the important transfer is visible flow and useful intervention.

Scope and reuse cautions. Fluids, electricity, unit pathfinding, multiplayer synchronization, modding, and a large campaign multiply complexity. They are mature-game features, not a suitable initial checklist. The repository identifies GPL-3.0 licensing; the complete asset inventory was not audited. An original implementation with original presentation avoids assuming every image, sound, or external dependency is covered by the same terms. Repository and license

2. osu!lazer - repeatable mastery supported by a creator ecosystem

What the repository actually represents. `ppy/osu` is the open source osu!lazer client, the successor to the older osu! client. Its repository popularity should not be described as an active-player count for lazer, or treated as a clean measurement of the whole osu! community. The README calls it a free-to-win rhythm game, documents desktop and mobile builds, and separates the game from the reusable osu!framework and resource repositories. The inspected client is principally C#/.NET, with shared game code and platform entry points. README

Core loop and decisions. In the standard ruleset, a player reads a sequence of circles, sliders, and spinners, positions the cursor, and times inputs to the song. A run produces a score and accuracy feedback; repeating a map makes personal improvement observable. The decision space is partly preparatory: choose an appropriate map, difficulty, and modifier, then decide whether to prioritize completion, accuracy, or a harder challenge. During play, the challenge is execution under timing and spatial demands rather than a large menu of strategic choices. The ruleset file exposes editor objects and modifiers including Easy, No Fail, Half Time, Hard Rock, Double Time, Hidden, Flashlight, and several experimental variations. Ruleset source

Why the loop can be fun - design inference. The same challenge remains available after failure, so players can distinguish improved skill from luck. Music gives actions a meaningful rhythm and helps players anticipate what comes next. Immediate judgment converts an imprecise feeling into a specific performance signal. Difficulty and modifiers let the player seek a challenge close to current ability. These features plausibly support mastery and repeated attempts; source inspection alone does not prove their contribution to retention. The broader ecosystem supplies an additional reason to return: new maps and music let familiar controls produce unfamiliar challenges.

How the implementation supports that experience. `DrawableHitCircle.cs` consults `HitWindows.ResultFor(timeOffset)` and handles late misses, hittability, approach-circle presentation, and visual responses. Timing windows make the relationship between an action and its result explicit. `OsuRuleset.cs` supplies factories for gameplay, scoring, health, beatmap conversion, and processing, giving each concern a named role. `OsuScoreProcessor.cs` adds ruleset-specific score handling, including rank treatment when misses occur. A lesson for Claude is to specify the judgment model before polishing effects, and to ensure that animation accurately communicates the state used by scoring. This inspection does not establish an end-to-end latency benchmark. Hit-circle source · Score processor

Popularity mechanisms supported by evidence. The project supports custom rulesets and community localization. The official Featured Artists program supplies licensed tracks and mapping resources, and describes mapping activities and community recognition. These are observable ways to lower creation

friction and reward participation. They support an ecosystem explanation; they do not establish which mechanism caused the observed 19,376 stars. README · Featured Artists

What Claude should learn. Build a small, reliable timing challenge; show an understandable result after each attempt; make restart quick; and provide a difficulty ladder that changes a few understandable dimensions. Separate song/map data from the renderer and judge inputs against an audio-based time reference. Treat calibration, clear feedback, and good challenge authoring as product requirements. Delay global rankings, multiplayer, a large editor, and broad ruleset support until one local mode feels fair. These are recommendations inferred from the architecture, not claims that a small prototype can reproduce osu!'s established community.

Free use and licensing. The client is free and its software is MIT licensed, with branding/trademark and resource qualifications documented by the project. Music rights are separate. The Featured Artists page describes licensing for use in osu!; that is not a blanket license to place those tracks in a new game. An original Claude-built game should use original, properly licensed, or synthesized audio and its own identity. README · Featured Artists terms

3. Lichess: make improvement part of playing

Repository: lichess-org/lila. **Observed GitHub interest:** 18,946 stars and 2,805 forks. **Category:** Free online chess platform, included as a service-design case rather than a standalone packaged game. **Free route:** lichess.org.

What the product actually does

Lichess combines chess play with puzzles, computer analysis, tournaments, studies, and other ways to learn or participate. Its official About page reports more than five million games played per day; this is the operator's claim about games, not a count of distinct daily players or a statistic derived from GitHub stars. The same page describes volunteer and donation support. About Lichess.

The important product sequence is play, encounter a mistake, inspect it, practice, and play again. That cycle supplies a reason to return beyond collecting a cosmetic reward. The inspected puzzle interface gives the loop a concrete shape: it maintains play, retry, and viewing states; wrong answers can lead to another attempt or examination of the solution; completion advances the session. Puzzle controller.

Why it can be fun and why people may return

Design inference: visible skill growth makes repeated play feel cumulative even though each new game resets the board. A mistake becomes a specific next task. Different time controls and puzzle activities also give players several ways to fit the same interest into different amounts of available time. Free access reduces the cost of trying those activities; public study and competition can turn an individual hobby into a shared one.

The official usage claim supports substantial activity, but it does not isolate which feature caused it. Chess already has a large cultural audience. Do not assume that adding ratings and puzzles to an unfamiliar game creates a similar audience automatically.

How it is made and what the code teaches

The README describes a Scala 3 server with a modified Play framework, a TypeScript web client, separate pure chess logic, a separate WebSocket server, and distributed engine analysis. The key transferable boundary is between rules, live delivery, presentation, and analysis. Its deployment architecture serves a large existing service and is excessive for a small prototype. Architecture overview.

`PuzzleSelector.scala` separates puzzle selection from presentation. It considers the session, requested angle and difficulty, already-played puzzles, missing records, and bounded retry paths. This illustrates that reliable content delivery includes recovery when the ideal next item is unavailable. Selection logic.

`PuzzleFinisher.scala` distinguishes rated and casual outcomes, updates player and puzzle performance, and treats repeat attempts differently from a new result. `moveTest.ts` checks the played sequence against the solution and handles special cases such as alternative castling notation. These are useful examples of keeping outcome rules explicit. Completion logic and move checking.

What Claude should use

Add a small review loop to an original game: show three important decisions, explain each from recorded events, and let the player retry one situation. In a sports game, a useful recap might expose an exhausted lineup facing a particular matchup. In a logistics game, it might highlight the edge where capacity failed. The useful lesson is that learning belongs inside the product.

Scope trap: copying the whole online platform introduces matchmaking, moderation, anti-cheat, accounts, and infrastructure before the new game's basic decisions have been proven. Begin with local play and honest analysis. Lila is AGPL-3.0-or-later according to its README; inspect its license and each separate dependency before any direct reuse. Studying the architecture does not require importing its implementation.

4. OpenRA - opportunity costs across economy, territory, and combat

What it is. OpenRA is an open source real-time strategy engine and a set of reimaginings of Red Alert, Tiberian Dawn, and Dune 2000. It is not an EA-endorsed continuation. The project combines C#/.NET engine code, composable gameplay traits, YAML definitions, Lua mission scripting, and platform/native components. Its README points creators to a mod SDK and documentation. These features make it both a playable project and infrastructure for related games. README · Mod SDK

Core loop and decisions. Gather resources, turn them into production capacity and units, scout, control useful space, and commit forces while protecting the economy that sustains them. Each expenditure removes another option: extra production may delay an army; a raid may expose a home base; concentrating forces may concede information elsewhere. Because opponents act simultaneously, a locally strong decision can become weak if its timing is wrong. Information and travel time make the map part of the economy rather than merely a backdrop. The inspected resource and production implementations support this description of linked constraints. PlayerResources.cs · ProductionQueue.cs

Why the loop can be fun - design inference. A player constantly revises a plan using incomplete information. Several usable strategies can coexist because resources, positions, counters, and timing interact. Mastery can improve through both faster execution and better judgment. Replays help transform a confusing loss into a specific lesson. The official project describes modern controls, fog of war, replay

support, and ongoing balance work; these are evidence of supported features, not proof that any one feature caused popularity. Official about page

Source-level architecture. `PlayerResources.cs` maintains synchronized resource state, tracks spending, and distinguishes available forms of resource payment. It also throttles insufficient-funds feedback. `ProductionQueue.cs` connects production to technology, ownership, resource availability, and power; it handles events such as sale, destruction, and ownership changes, including queue cleanup and refund behavior. Production is therefore a stateful process linked to the world, not an isolated countdown. These are useful examples for Claude of making rules composable while explicitly handling transitions. Resources source · Production source

Why multiplayer is a separate engineering milestone. `OpenRA.Game/Network/OrderManager.cs` coordinates frame-numbered orders from clients, checks whether the required packets have arrived, compares synchronization state, and supports replay-related execution. That machinery explains why a convincing online RTS requires more than adding a socket to a local prototype. The systems must agree on how commands change the simulation, and divergences need useful diagnostics. Claude should first prove one offline match, readable unit behavior, and a coherent economy. Determinism, network recovery behavior, replays, and competitive integrity require their own scope and validation. OrderManager.cs

Evidence behind community appeal. OpenRA offers familiar classic-game premises alongside current-platform support, modern controls, community maps, a mod SDK, and documented extension points. Familiarity may reduce learning friction, and creator tools may extend content supply; those causal links are design hypotheses. The observed 17,737 stars and 3,111 forks demonstrate repository interest, not online-player totals. Its official download page listed both the March 2025 stable release and an October 2026 playtest, providing direct evidence of a newer testing channel. About · Downloads · Community maps

What Claude should learn and avoid. Build a small economy whose spending choices are visible, a few units with clear roles, and a map where distance and scouting matter. Use a modest set of composable behaviors and validate ownership/destruction transitions. Avoid starting with multiple factions, large campaigns, unrestricted unit counts, or online matchmaking; pathfinding and balance can dominate even a small RTS.

Free play is separate from asset reuse. OpenRA's engine is GPL-3.0-or-later. Its official download guidance states that the included classic-game mods require original game files obtained separately, with installation assistance to download or copy the needed assets, and references EA's C&C Franchise Modding Guidelines. Those original assets are **outside OpenRA's software license**. Availability through an assisted installation is not a general right to redistribute them in a new game. A Claude-built original should use its own or independently licensed artwork, audio, names, and world. README license notice · Official asset and download guidance

5. Cataclysm: Dark Days Ahead - stories emerge from interacting constraints

What it is. Cataclysm: Dark Days Ahead is a turn-based survival game in a persistent, procedurally generated post-apocalyptic world. Its README describes scavenging for supplies, equipment, and vehicles, and surviving threats that include zombies, insects, robots, and other humans. The project is principally C++, supports terminal and SDL tile presentations, and distinguishes stable builds from frequent experimental development. The latest stable release observed was 0.I-1 on September 20, 2026, while the default branch had activity on November 5. These are separate release channels, not a single stability promise. README · Stable release

Core loop and meaningful decisions. Identify an immediate need or a longer-term goal, find a route to useful resources, judge the risks, act, and manage the resulting condition of the character and equipment. Crafting extends the loop by turning collected materials, tools, knowledge, and safe working conditions into new options. Progression therefore includes both possessions and capability: obtaining an item may matter because it enables a new plan. The player can set goals within the survival framework, which creates space for a personal sequence of problems rather than one prescribed route through content. README · Crafting implementation

Why the loop can be fun - design inference. Interacting rules produce situations the designer did not need to script individually. A trip for one resource can expose another opportunity or create an injury that changes the next goal. The resulting sequence feels like a story because the player supplied the intention and lived with the consequences. The cost is complexity: a simulation can generate tedious chores or obscure failure conditions as easily as memorable stories. A useful design test is whether a subsystem creates a new choice or merely adds repeated upkeep. This evaluation follows from the design; no player-retention experiment was conducted here.

How the source makes crafting a situational decision. `src/crafting.cpp` checks more than whether a material list is present. The inspected code considers morale, lighting, driving or other context, relevant abilities, and manipulation constraints. Negative morale can penalize crafting; positive morale is not simply an unlimited productivity bonus. Darkness interacts with recipe flags and proficiency or skill conditions. Crafting consequently asks where, when, and in what condition a task can be performed. For Claude, the lesson is to connect a few systems through explicit rules, then expose the relevant reason when an action is blocked or slowed. crafting.cpp

How content is represented. `src/recipe.cpp` loads structured recipe definitions covering skills, proficiency, learning routes, activity levels, requirements, byproducts, and construction or disassembly relationships. It handles both inline and referenced requirements, and enforces constraints such as incompatible combinations of reversibility and byproducts. The architecture allows a common system to interpret many definitions. This is evidence for separating content data from core behavior, while also validating that data. It is not evidence that every new item is cheap to balance: each additional rule can interact with existing resources and progression. recipe.cpp

Evidence behind popularity and persistence. The README describes a long-running volunteer project and attributes contributions to more than a thousand volunteers. Stable and experimental channels, contributor documentation, and data-driven content give the community multiple ways to participate. The observed 13,468 stars and 4,719 forks are consistent with considerable repository engagement, but they cannot tell us how many people play or whether simulation depth caused that engagement. Avoid converting the README's contributor statement into an independent measurement. README

What Claude should learn and avoid. Use a small network of mutually relevant needs, items, and actions. Make blocked actions explain themselves. Let players set a goal, assemble a plan, and face a consequence that changes the next plan. Keep recipe definitions separate and validate references. Limit early scope to enough systems to create a few surprising interactions; reproducing a vast survival simulation, vehicle system, and content catalog would swamp a first playable build.

Free availability and the actual license. GitHub's repository API returned `NOASSERTION` for license recognition. The inspected `LICENSE.txt` explicitly identifies **Creative Commons Attribution-ShareAlike 3.0** for the main project and lists separate third-party terms, including font and library licenses. `NOASSERTION` must not be reported as no license. Free downloads and source availability do not remove attribution, share-alike, or component-specific obligations. An original implementation can learn from the rules without copying protected content. LICENSE.txt

6. 2048: a small ruleset with persistent spatial consequences

What the evidence establishes

2048 is a free browser puzzle with mobile editions linked from its README. Its author describes influences from 1024, Saming's 2048, and Threes, which matters when tracing the design's lineage. The repository is MIT licensed. Its README credits contributions to mobile swipe controls and persistent best scores. These are concrete signs that distribution and usability grew beyond the initial implementation. README · play · license

The entry point constructs a four-by-four GameManager with injected keyboard input, HTML presentation, and local storage managers. Grid stores cells, checks bounds and occupancy, chooses a random empty cell, and serializes state. This is a compact JavaScript/HTML/CSS application, not a large engine-dependent production. application.js · grid.js

The manager starts with two tiles. A direction slides all tiles toward an edge; matching neighbors combine, and a newly merged tile cannot merge again during that move. A successful merge adds the new value to the score. Only a move that changes the board spawns a tile: a 2 with 90% probability, otherwise a 4. The traversal order follows the movement direction. Reaching 2048 records a win while allowing continued play. A full board ends play only when no adjacent equal tiles remain. The manager restores saved games, updates the best score, and clears the current game after a loss. game_manager.js

Core loop and meaningful decisions

The core loop is read the board, choose a direction, observe simultaneous movement and merges, then reassess after the new tile appears. A single input has consequences across the whole board. The player is choosing a future arrangement as well as taking the immediate merge.

The most useful design lesson is spatial opportunity cost. Empty cells are a resource even though the interface does not label them that way. A large merge is rewarding, but it can produce a position that is harder to develop. Keeping related values near each other, preserving maneuvering room, and choosing where the largest tile belongs create strategy from a handful of rules. Random spawns interrupt perfect planning, yet the movement rules remain stable enough that players can improve.

Why the design may be fun

This is an inference from the rules, not evidence about individual players. The game makes progress visible: higher numbers occupy the same small space. Feedback is immediate, and the next decision appears as soon as the board settles. A clear target reduces the explanation needed before a new player acts.

Failure can also be legible. The board visibly becomes crowded, and players can often imagine a different earlier move. That supports a plausible desire to retry. Local best scores create a personal comparison without requiring accounts or a competitive service. The same deterministic movement rule repeatedly produces unfamiliar positions, supporting replay without hand-authoring levels.

There is a tradeoff: randomness can also make a loss feel unfair, and repeated runs can become repetitive once the player finds a comfortable strategy. A successor should preserve meaningful consequences while giving players another reason to reconsider their habits.

Plausible popularity mechanisms

The observed repository has more forks than stars. Combined with its MIT license and compact architecture, that supports the interpretation that it became an unusually reusable programming example and remix base. It does not establish how many people played it. A free URL, swipe input, simple presentation, and a rule that fits in one sentence plausibly reduce the friction of trying and sharing the game. These mechanisms should be distinguished from any claim that one feature caused its success.

What Claude should borrow

Borrow the small decision space, visible constraint, and precise state transition. Implement movement as a pure board transformation before adding animation. Make the random generator injectable so a seed can reproduce a run. Separate legal moves, merging, scoring, spawning, and terminal-state detection. The original code's separation of state, input, presentation, and storage is useful even if the new implementation uses a different framework.

Meaningful verification should cover directional traversal, the single-merge-per-turn invariant, blocked moves that do not spawn tiles, and correct loss detection. Keep the saved state versioned. Add a readable motion-reduced mode and input instructions without making players open a help page.

The scope trap is producing a reskin with no different decision. Another is adding many special tiles before the basic board is satisfying. One well-understood complication is enough for a first prototype.

7. Unciv - connect the next turn to a long-term identity

Premise and availability. Unciv is a free, open-source, Civilization V-inspired turn-based strategy game for Android and desktop. Exploration, settlement, production, research, diplomacy, and warfare compete for attention and resources. Its README emphasizes speed, small size, modding, and numerous distribution channels, including mobile stores and desktop package systems. A documented Docker/VNC route permits remote browser access; that does not make the game a native browser implementation. README

The decisions that create the game. Every investment has an opportunity cost. A city that produces a military unit delays a building; expansion creates future income while demanding defense; research shapes later options. The map and rival civilizations make a generally sensible plan insufficient: the player must respond to this situation. Short milestones, such as finishing construction or research, coexist with an increasingly distinct civilization. The turn structure gives the player time to compare these choices before committing.

Design inference: why this can be fun. Several visible projects are always nearing completion, making another turn attractive. Decisions also accumulate into an identity: a research direction, city network, or diplomatic position. Replay comes from different geography, rivals, and choices interacting with the same legible rules. This is not evidence that more systems automatically improve a game. The transferable pattern is a few short progress clocks nested inside a longer plan, with enough conflict that the player cannot advance every clock equally.

How it is made. The implementation uses Kotlin and LibGDX, with core game logic separated into classes for game state, cities, civilizations, rules, and automation. Three inspected files show why that structure matters:

- `GameInfo.kt` coordinates `nextTurn`, civilization rotation, global turn advancement, human-player handoffs, and automated turns. AI civilizations pass through explicit start, automation, and end phases. An authoritative turn coordinator makes recurring effects and ownership changes easier to reason about.

- `CityConstructions.kt` validates the construction queue, compares accumulated production with cost, handles completion, and manages overflow. It caps overflow using construction cost or current production rather than letting an arbitrary surplus become an unlimited exploit. A unit that cannot be placed can fail completion and generate a notification. The edge cases are part of the game economy, not merely technical cleanup.
- `NextTurnAutomation.kt` sequences AI work such as handling popups and trade, then diplomacy including war, peace, and friendship. This is a concrete example of an opponent operating through structured game systems instead of a separate decorative simulation.

Popularity and reach. The project offers a familiar strategy premise with free access and a comparatively lightweight presentation. Mobile availability, desktop packages, a tutorial, the Civilopedia, translations, modding, and community channels create documented routes into the game and into contribution. These are plausible explanations for reach and persistence, not a measured attribution model. The repository's stars and forks demonstrate public interest in the project, not the size of the paying or active audience. README

What Claude should learn. Use a single turn pipeline and a transparent resource ledger. Define when income, costs, training, construction, and opponent choices occur. Show what will finish next turn and what the player gives up by selecting it. For One More Season, a short season with roster and training decisions can reproduce this planning tension within a much smaller ruleset. Opponents can use the same legal actions with a small set of priorities.

Scope and reuse cautions. Strong AI, a large rules vocabulary, diplomacy, map generation, and explanatory interfaces are substantial projects. Start with a few resources, a short campaign, and readable rival behavior. Code is primarily MPL-2.0, with documented exceptions. Assets have mixed Creative Commons and public-domain terms; the credits identify attribution and share-alike obligations. Do not treat source availability as permission to copy another game's names, art, text, or branding. Credits · Repository

8. OpenTTD - let players program a useful network

Premise and availability. OpenTTD is an open-source transport simulation based on Transport Tycoon Deluxe. Players connect producers, towns, and destinations with transport services, buy vehicles, assign orders, earn delivery income, and improve the network. Official releases are freely available through the website and other distribution channels. It requires graphics, sounds, and music; free replacement sets such as OpenGFX, OpenSFX, and OpenMSX are separate projects with their own licensing. README · Official website

The decisions that create the game. A route must do more than exist. It must carry the right goods to a destination that accepts them, using a service with workable capacity and timing. Vehicle purchases compete with infrastructure spending. Distance, speed, reliability, and congestion interact. Once a network is running, the player can discover a specific weakness, make an improvement, and observe the result across many subsequent trips. Repeated automated activity lets a single design decision have a lasting visible effect.

Design inference: why this can be fun. The world becomes a record of the player's decisions. A vehicle arriving successfully is feedback on a route, a timetable, an investment, and earlier planning. Networks support self-directed goals, from building something profitable to making a complicated service run smoothly. Multiple reasonable solutions allow personal style and optimization. The key transferable idea is persistent, inspectable consequences: the player should see the system execute a plan and understand why performance changed.

How it is made. The game uses C++ and CMake, with platform-specific graphics and application support. The repository documents save compatibility and extensive content customization, including NewGRF. Those capabilities reflect a long-lived project and should not be confused with the minimum requirements for a new transport game. Two inspected implementation files expose particularly useful design principles:

- `src/economy.cpp` calculates cargo income from factors including quantity, distance, the cargo's rate, and transit time. It supports alternate cargo behavior through NewGRF callbacks. Delivery logic checks acceptance and updates the economic result for accepted cargo. The reward therefore recognizes delivered service, not merely building track or moving an empty vehicle. This aligns the player's financial objective with a useful network outcome.
- `src/order_cmd.cpp` implements station, depot, waypoint, and conditional orders. It supports service-if-needed behavior and searching for a depot. Conditional order evaluation has a recursion bound tied to the number of orders, protecting the game from a looping instruction sequence. Order processing distinguishes traveling, loading, and leaving states and updates interface state when orders change. The player-facing itinerary is a constrained programming language backed by defensive execution rules.

Popularity and reach. Official downloads, package availability, multiplayer, community forums, documentation, translations, and user-created content provide many ways to discover, learn, and extend the game. Compatibility and customization can support a durable community because earlier work remains useful, although the source inspection does not quantify their effect on retention. Stars measure interest in the repository; the much broader distribution ecosystem means they cannot be read as a direct player count. README · Wiki

What Claude should learn. Give the player a small set of typed instructions, let the system execute them, and make execution observable. Reward completed useful work, account for recurring costs, and provide clear reasons for failure. In a sports management setting, fixture travel or facility scheduling can use this structure without implementing railway signaling. A route or schedule preview should explain expected arrival, capacity, and cost before the player commits.

Scope and reuse cautions. General pathfinding, rail signals, deadlocks, dense world simulation, robust multiplayer, and decades of save compatibility are major scope traps. A small prototype can use a fixed graph and a handful of vehicles while retaining meaningful routing decisions. The README and copying file identify GPL v2 licensing; many source headers specify v2 or later, and bundled third-party material has exceptions. GitHub's metadata returning `NOASSERTION` does not mean the project is unlicensed. Inspect the actual copying file and each reused dependency. COPYING.md

9. A Dark Room: progression changes the kind of attention required

What the evidence establishes

A Dark Room describes itself as a minimalist text adventure and offers a free browser version alongside other editions. Its README lists 23 languages. The source is licensed under MPL 2.0. The package file identifies JavaScript tooling, an Express development server, and localization tooling; the browser game itself does not require players to operate a Node server. README · play · package.json · license

Room defines timer-driven fire and character states, craftable items, costs, messages, and limits. Helping the stranger eventually enables a builder income stream of two wood per ten-second interval. Forest access, new interface elements, and character changes follow state conditions. Buying a compass sets the

path-discovery flag. The document title changes as the fire changes. Crafting recipes progress from traps and huts to workshops, industrial buildings, equipment, and weapons. room.js

Outside defines jobs as resource transformations. Hunters produce meat and fur; tanners consume fur for leather; charcutiers consume meat and wood for preserved food; miners consume preserved food to produce ore. Huts determine population capacity. The code also handles arrivals, worker assignment, trap results, and losses. outside.js

World defines an ASCII map with terrain, fog, landmarks, combat, and expedition-related resources. Constants include food, water, health, and weapon costs. Landmarks have distance constraints: progressively different resources and locations appear farther from the village. Cleared dungeons can become outposts connected by roads. The map and discovery state persist. These observations concern inspected definitions and initialization; the entire movement and combat implementation was not audited. world.js

Core loop and meaningful decisions

The early loop is perform a simple action, notice a change, and discover another action. Later, workers and buildings transform resources. The expedition layer turns that economy into preparation for travel, while travel returns resources and discoveries that enable further development.

The design changes what the player pays attention to. At first, the fire and stranger matter. Later, allocations and production chains matter. Eventually, distance, supplies, combat readiness, and returning safely matter. These are connected stages rather than an unstructured list of features.

The inspected production recipes create opportunity costs. Diverting workers or raw resources to one output limits another. Growing population increases productive capacity, but capacity alone does not remove bottlenecks. Expedition preparation gives economic growth a purpose beyond watching a number rise.

Why the design may be fun

The central inference is that curiosity is rewarded with a change in agency. A reveal does more than add prose: it changes what the player can do. That makes the discovery valuable even if the player is not primarily motivated by narrative.

Automation can preserve novelty when it arrives at the right moment. A task first teaches the player a resource's importance. A helper then takes over some of the repetitive work, freeing attention for a larger decision. The relationship between learning, automation, and expanded scope is more useful to borrow than any particular waiting timer.

Minimal presentation may also increase the role of imagination. A short description can imply a world that would be expensive to render. That is an inference about the form's affordances, not a universal preference. Some players will find timer-driven opening sections slow or the interface opaque.

Plausible popularity mechanisms

A browser link and restrained visual requirements plausibly make the game easy to try. The documented localization work extends accessibility across languages. Its changing scope may encourage recommendations based on surprise, although this research did not measure sharing or retention.

The repository's 8,380 stars show substantial developer attention, not a count of players. Longevity and multiple editions are evidence that the work exists across channels, but they should not be converted into sales or engagement estimates without separate sources.

What Claude should borrow

Use a capability map. Each milestone should unlock a new verb, choice, or way of allocating attention. Define exactly which state condition triggers each reveal, then record the reveal so reloading cannot replay or duplicate it. Keep narrative text separate from economy calculations. A production recipe should declare inputs, outputs, interval, and staffing requirements.

The inspected architecture uses a shared state manager, an engine coordinator, module objects, and state-update events. The transferable principle is coherent state ownership. A new game should avoid spreading direct mutations through presentation components. Its save format should distinguish resources, capabilities, narrative flags, map discoveries, and assignments.

For a small prototype, reduce waits. The first meaningful change should arrive during the initial session. Do not imitate the full content volume, extensive travel layer, and many equipment tiers at once. A mystery opening is only valuable if the later decision is worth discovering.

MPL-covered code and separately sourced assets need their applicable terms checked before reuse. An original implementation can borrow the progression premise without importing the original text, world, or interface.

10. Endless Sky - make growth change both freedom and responsibility

Premise and availability. Endless Sky is a free, open-source space trading and exploration game. The player accepts transport or other work, travels between systems, trades, encounters combat, upgrades ships, and can pursue story missions. Its official description supports different identities, including merchant, explorer, and bounty hunter. Releases are available through GitHub and major desktop distribution channels. Narrative progression coexists with open-ended activity. README · Official website

The decisions that create the game. Jobs consume capacity, time, and exposure to risk. Cargo space, passenger bunks, crew, equipment, and destination requirements prevent the player from accepting everything. Earnings can improve a ship or grow a fleet, but ownership also brings ongoing obligations. An upgrade matters when it changes which opportunities can be attempted. Missions and encounters provide reasons to travel beyond a simple buy-low, sell-high loop.

Design inference: why this can be fun. A humble starting position provides a clear comparison with later capability. The player can imagine a role and work toward equipment that makes that role practical. Capacity constraints create understandable compromises, while story gives transactions a human context and interrupts repetitive earning. Progress is more interesting when it expands choices and introduces new costs. A larger fleet should feel different to manage, not merely show a larger number on a balance sheet.

How it is made. The inspected build specifies C++20, CMake, OpenGL, and SDL support, with SDL2 as the default and SDL3 optional. The repository separates mission logic, persistent player state, simulation, rendering, and data. Three inspected files provide concrete patterns:

- `source/Mission.cpp` separates whether a mission can be offered, accepted, and completed. Checks include locations, conditions, repeat rules, available cargo or passenger capacity, action feasibility, destinations, waypoints, NPC conditions, and other requirements. Required cargo or passengers cannot simply be left on a remote ship. The interface can consult the same satisfaction logic as completion, reducing contradictions between what a map implies and what the rules permit.

- `source/Engine.cpp` separates paused and active calculation, processes AI and player commands, moves ships, and prepares draw, radar, and other buffers. This supports a distinction between world state and its presentation. The inspected section alone is not sufficient to make claims about the full threading model or deterministic simulation.
- `source/PlayerInfo.cpp` advances accounts using assets, salaries, and maintenance costs, with messages related to salary and mortgage obligations. Growth therefore interacts with operating expenses. Persistent ownership and recurring consequences belong in the player model, rather than being improvised by individual screens.

Popularity and reach. Free desktop distribution, GitHub releases, Steam, community documentation, and opportunities to contribute art, writing, and code are documented avenues for participation. A content-driven game can welcome contributors with different skills, potentially expanding variety. That is a plausible mechanism, not proof that content contributions explain its stars or long-term player engagement. README · Player manual

What Claude should learn. Define jobs as data with explicit offer, acceptance, success, and failure conditions. Use shared predicates in rules and interface explanations. Make capacity constraints visible before acceptance. Let upgrades unlock new activities, and attach understandable recurring costs to expansion. In a management game, contracts, trips, or tournament invitations can create this same pattern without space combat. Narrative events should change a relevant choice or constraint rather than merely interrupt play.

Scope and reuse cautions. Real-time combat, ship physics, fleet AI, a large universe, and branching story content create separate burdens. A small contract game can retain the economic and narrative decisions on a fixed map. Code is GPL-3.0, with source headers permitting v3 or later. Assets use mixed public-domain, Creative Commons, and other terms recorded in the copyright inventory. Audit individual assets before reusing them. Build configuration · Copyright inventory

11. Shattered Pixel Dungeon - constrained randomness and decisions that cost time

What it is. Shattered Pixel Dungeon is a traditional roguelike derived from Watabou's Pixel Dungeon, with randomized levels, enemies, and a large item collection. Its README links Android, iOS, and desktop distribution channels, including GitHub releases. The code uses Java with Gradle and libGDX; the inspected build configuration declares Java 11 compatibility and libGDX 1.14.0. Platform support does not mean every storefront version is free. GitHub releases provide a free route to supported builds; individual storefront offers must be checked separately. README · Build configuration · v4.0.2 release

Core loop and decisions. Explore a floor, evaluate threats and resources, move or act, resolve the consequences, then continue deeper if the character survives. Positioning, consumables, equipment, and the decision to engage or retreat compete for scarce resources. The turn-based structure provides room to think, while action costs keep individual decisions consequential. Randomized environments change the immediate puzzle; learning the relationships among enemies, items, and space carries across attempts. This is a useful model of replay value based on increased player understanding rather than simply increasing a permanent numerical reward. README · Actor scheduling

Why the loop can be fun - design inference. A limited supply of useful items makes both spending and saving them meaningful. Uncertainty asks the player to form a hypothesis, accept a risk, and interpret what happened. A new layout disrupts rote movement while familiar rules preserve the value of expertise. Failure can feel instructive when the cause is understandable and the next run offers another opportunity to apply

the lesson. Conversely, unexplained interactions and excessive early complexity can make the same design feel arbitrary. Claude should preserve readable consequences while limiting the initial number of systems.

Source-level lesson: randomness needs structure. `RegularLevel.java` assembles authored room types within randomized layouts and includes generation, connection, and painting checks rather than accepting every arbitrary arrangement. Entrances, exits, and room budgets constrain the output. `Generator.java` uses weighted item categories, tracks depletion and reset behavior, and distinguishes special item pools. It includes a fallback when artifact availability is exhausted. These mechanisms show that procedural variety can be shaped by budgets, guarantees, and supply rules. The transferable lesson is to generate situations with a purpose, not merely scatter content with independent random rolls. RegularLevel.java · Generator.java

Source-level lesson: time is a gameplay resource. `Actor.java` schedules actors by their accumulated time and resolves ties through priority. Actor state includes time and identity for restoration. The scheduler also coordinates with sprite movement and waits for visual completion. It therefore supports an explicit action-cost model, but should not be represented as a fully isolated simulation with no rendering coordination. A small game can borrow the clear costs and deterministic ordering without copying this threaded scheduling machinery. Actor.java

Evidence behind community appeal. The repository documents releases, translation participation, and a guide for creating forks. It does **not** accept pull requests, so its appeal should not be explained as unrestricted community code contribution. The observed 6,764 stars and 1,720 forks indicate substantial repository interest; the available sources do not isolate the effects of mobile access, localization, the parent game's audience, or design quality. README · Fork guidance

What Claude should learn and avoid. Start with a small set of enemies and consumables whose interactions are distinct and explainable. Give every action a visible cost, guarantee viable generated layouts, preserve a run seed for debugging, and make death explainable. The main scope trap is the multiplication of interactions among items, statuses, enemies, and map features. The project is GPL-3.0 licensed; adapting its code requires observing the applicable copyleft obligations. Its fork guide also calls for changing the name, icon, package identity, and update/news endpoints, and retaining appropriate credit. Original design inspiration does not require copying either code or branding. License · Fork guidance

12. The Evolution of Trust: make the player's prediction the hook

Repository: ncase/trust. **Observed GitHub interest:** 6,339 stars and 1,007 forks. **Category:** A finite playable explanation and simulation, included for its unusually transferable onboarding and discovery design. **Free route:** ncase.me/trust.

What the game actually does

The player encounters a small cooperation game, tries repeated interactions, predicts which strategies will succeed, and watches tournaments and evolving populations. Later sections change assumptions such as the number of interactions, payoffs, or mistakes. A sandbox lets players manipulate those conditions. This is a designed sequence toward understanding, rather than an endless progression game. Narrative and interaction text.

Why it can be fun and why people may share it

Design inference: committing to a prediction creates personal investment before the explanation arrives. The result can confirm an intuition or expose a missing assumption. Short rounds make experimentation cheap, while recognizable characters give abstract strategies a memorable identity. The reward is a better explanation of what just happened.

The repository documents a large set of translations and a deliberately separated text file. Those are visible mechanisms for accessibility and sharing across language communities. They do not provide a verified number of players or establish that every reader learned the intended lesson. The original experience is a strong reference for games about incentives, negotiation, teamwork, or resource allocation. Project and translation documentation.

How it is made and what the code teaches

The project uses JavaScript, HTML, and CSS, with documented rendering, sound, animation, and message-passing libraries. The implementation separates a reusable simulation from the instructional sequence.

`PD.js` defines the payoff matrix, repeated games, round-robin tournaments, strategy memory, and optional action noise. In its default payoff structure, mutual cooperation yields two points to each player, unilateral defection yields three to the defector and minus one to the cooperator, and mutual defection yields zero. The numbers are model assumptions, not empirical laws of social behavior. Simulation rules.

`Tournament.js` handles agents, tournament progression, eliminating low-scoring agents, and reproducing successful strategies. `6_Slides_Noise.js` controls a teaching sequence that introduces mistaken actions and changes what the player should expect. `Slideshow.js` supplies lifecycle hooks for moving between authored stages. This is a useful separation of simulation, presentation, and lesson choreography. Tournament, noise chapter, and stage controller.

What Claude should use

Ask for a prediction, run a transparent model, show the consequence, then change one assumption. This structure is especially suitable for an original household-budget or team-cooperation game. Use a saved seed to make comparisons interpretable; the inspected original contains `Math.random()` calls, so reproducible experiments are a proposed improvement for the new project rather than a feature being attributed to this code.

Scope trap: a polished animation can make a narrow model sound like universal truth. Expose assumptions and show conditions under which an apparent winner stops winning. The repository's overall dedication is CC0, but its README explicitly lists some third-party sounds with other terms, including CC BY-NC and CC Sampling+. Replace or individually clear those assets before direct commercial reuse; the repository badge is not a blanket rights clearance.

13. Bitburner: automation becomes both the tool and the reward

What the evidence establishes

Bitburner is a programming-based incremental game with a hacking setting. Its current README links a free browser release, a development build, and a Steam edition. Documentation is integrated into the game, with API declarations and generated references also present in the repository. The historical repository explicitly redirects development to bitburner-official/bitburner-src. current README · free browser release · migration notice

The inspected development package is version 3.0.2 and uses TypeScript, React, Webpack, Monaco, and Electron, with testing and documentation tooling. The latest GitHub release retrieved was stable 3.0.1, published 17 May 2026. The development branch and released game should not be assumed identical. package.json · 3.0.1 release

Hacking centralizes formulas for success chance, experience, stolen money, and operation duration. Difficulty, player skill, and multipliers affect those outcomes. Growth takes 3.2 times the hack duration, while weakening takes four times the hack duration. Hacking.ts

NetscriptWorker compiles and runs scripts through a defined API namespace, requires an exported main function for the newer script format, and manages worker registration, completion, crashes, and shutdown. A script's RAM cost is multiplied by its thread count and checked against available server memory before launch. PrestigeWorkerScripts stops all workers and clears ports. NetscriptWorker.ts

Prestige has separate augmentation and Source File reset paths. These explicitly handle scripts, servers, programs, player state, factions, persistent bonuses, and other systems. Their preservation rules differ. NetscriptFunctions assembles the large callable surface from specialized modules. Prestige.ts · NetscriptFunctions.ts

Core loop and meaningful decisions

The player writes behavior that creates ongoing output, then invests that output in more capability. The design adds a planning layer because automation consumes limited resources. Available RAM, threads, targets, operation timing, and server conditions determine what can run effectively.

The time relationships between operations matter. A script that pursues immediate extraction without maintaining the conditions for future production can be less useful than a coordinated routine. The player is improving a system rather than only improving a character statistic.

Reset systems create another decision: when is the future benefit worth giving up present capacity? The inspected code shows that reset is not merely assigning money to zero. It is an explicit transition with many different categories of loss and persistence. The game must communicate that contract clearly for the decision to feel informed.

Why the design may be fun

This is design inference. A working program feels like a tool the player made, so efficiency can become a personal achievement. Progress is partly intellectual: a better routine changes the results without requiring faster manual input. A player can recognize a bottleneck, form a hypothesis, implement a change, and see the effect.

The constrained resource model makes automation a strategic choice. Unlimited free execution would remove much of the allocation problem. RAM gives abstract code a visible opportunity cost. Operation timing creates a reason to understand a system rather than repeatedly press the most profitable-looking button.

The educational appeal of practicing programming is plausible, but this research did not measure learning outcomes. The same depth can create a barrier for new players. A large API and many independent subsystems can overwhelm someone before they experience the satisfaction of their first useful automation.

Popularity evidence and interpretation

Current-repository stars alone understate this game's historical GitHub footprint because development moved repositories. The historical and current totals must remain separate. Neither is a player count, and summing them would double count an unknown overlapping audience.

The free browser build plausibly reduces installation friction, while integrated documentation supports experimentation. Sharing scripts and strategies is a plausible community mechanism because the player's solution is an inspectable artifact. These are explanations to test, not measured drivers of adoption. Any review counts or storefront ratings should be sourced separately and dated rather than inferred from GitHub.

What Claude should borrow

Start with a deliberately small programmable world. Provide a documented API, deterministic calculations, a resource budget, visible results, and actionable error messages. Make a player's first script useful in a few minutes. Prefer a constrained command language or block editor for a small prototype; supporting arbitrary JavaScript creates runtime and debugging work beyond the core game.

Model each automated job as a worker with identity, resource cost, lifecycle, and cancellation. Show resource availability before launch. Preserve the script or command sequence in the save. Keep simulation calculations separate from the editor and renderer.

A reset should be a transaction: stop workers, apply the declared persistence policy, initialize the new state, and save it coherently. Show a before-and-after list to the player. Test that repeated load or reset cannot duplicate rewards or leave workers producing in a discarded world.

The main scope trap is importing the reference game's entire menu of corporations, gangs, factions, markets, contracts, and other subsystems. First prove one interesting programmable loop. Add another system only when it creates a new decision that existing tools cannot already express.

License caveat

The repository's license is Apache 2.0 with Commons Clause 1.0, including a restriction on selling the software as defined in that clause. Do not label it ordinary permissive Apache or assume all commercial reuse is allowed. GitHub reports its license as Other. The practical handoff instruction is to create original code and assets; review the actual terms before copying or incorporating this implementation. license.txt

Supplemental design note: shapez

tobspr-games/shapez.io had 7,003 stars and 1,310 forks in the snapshot. Its README links a Steam edition and an online demo, describes cutting, rotating, merging, and painting shapes, and provides local build instructions. Because public source, the demo, and normal full-game distribution are different access routes, it is not treated as an unqualified free full-game entry in the main table.

Design inference: a complicated target becomes approachable when the player can decompose it into small transformations. A functioning production line then makes their reasoning visible. This is a useful reference for teaching mechanics through composition: introduce one operation, create a reason to combine two, and later expose throughput as a new problem.

Lesson for Claude: in an original logistics game, make each transformation or routing rule legible and testable on its own. Then build goals that require combinations. For Blackout Dispatch, connectivity should be understood before capacity and redundancy are introduced. The proposed game uses its own rules and assets rather than reproducing shapez's machines, targets, or visual identity.

This is a supplemental premise comparison based on project documentation and repository metadata, not an additional complete source audit. Project README and build instructions.

What Claude should build from these lessons

Recommendation: build one small, complete game first

For Kevin, the best fit is **One More Season**, a short sports management game built around six consequential fixtures. For a broader audience, the best first experiment is **Blackout Dispatch**, a compact logistics puzzle about keeping a city powered through a storm. These are working titles and original design proposals, not researched games or verified unused names.

The recommendation is based on scope and the decisions the player gets to make. One More Season gives Kevin room to use his coaching experience to make tradeoffs credible. Blackout Dispatch has a visible board, a clear emergency, and a result that can be understood without sports knowledge. Neither requires an online population, a large asset library, or real-time AI-generated content to become enjoyable.

Do not start all the concepts at once. Complete one run from title screen to a satisfying ending, then observe people playing it. The research does not establish that any new concept will become popular.

The design principles worth carrying forward

1. Give the player a useful decision before a long explanation

2048 makes a board move understandable immediately. A Dark Room begins with a small action space and expands it. The transferable principle is to teach one decision by letting the player make it. Start with a visible objective, a small set of controls, and feedback that answers, "What did my action change?"

For a management game, the first screen should offer a consequential lineup or resource decision. A dozen dashboards, an empty inbox, and a character biography can wait. Within the first minute, a new player should be able to point to something they caused.

2. Make improvements visible in the world

Mindustry and OpenTTD connect a design change to something that moves, gets supplied, earns money, or stops working. Numbers matter because players can see what produces them. In a new game, show the chain: a short route frees capacity; rested players execute longer; choosing a heavy cargo contract leaves less room for fuel.

An upgrade should change a decision, an interaction, or an observable outcome. A succession of percentage bonuses can support a good loop but cannot substitute for it.

3. Build tension from competing good uses of scarce resources

Resources are interesting when more than one reasonable option needs them. An expedition's final battery can fund a detour, an emergency escape, or a lucrative delivery. A team's limited practice time can improve execution or reduce fatigue. Explain the tradeoff before the commitment and resolve it consistently afterward.

Design at least two credible strategies for each early scenario. If every player should make the same purchase whenever they can afford it, that purchase is primarily progression bookkeeping rather than a meaningful choice.

4. Let players improve their understanding, not just their inventory

Shattered Pixel Dungeon makes situational knowledge valuable. osu! exposes performance as a skill that can improve. Lichess surrounds play with analysis and practice. A successful run should teach something the player can use on the next attempt even when every upgrade is removed.

For each proposed game, define one sentence a player might say after losing: "I committed too much fuel to the short route," or "I played my strongest lineup while it was exhausted." If players can only say "the game gave me bad numbers," improve information and feedback before adding content.

5. Use surprise to reveal a new possibility

A Dark Room demonstrates how a familiar action can acquire new meaning as the game opens up. The Evolution of Trust shows how changing one assumption can overturn a prediction. Surprise works best when it makes the player reconsider a choice they understand.

A useful reveal might be a new type of contract, a new opponent behavior, or a second destination for an existing resource. Avoid revealing five disconnected management systems at once. Discovery should expand agency without making earlier learning irrelevant.

6. Make failure informative and restarting easy

A quick restart has value when players know what they want to try differently. Preserve the seed, identify the decisive choices, and offer an immediate replay. Distinguish a poor decision from a reasonable gamble that lost. A recap should show what actually happened, not invent a confident counterfactual.

If the game estimates an alternative, label it as an estimate. For a strict replay, reuse the saved state, rules version, and random stream. Do not compare two independently randomized outcomes and tell the player that the difference proves one decision was better.

7. Add content through rules that combine

Unciv, Cataclysm, and Endless Sky demonstrate different ways to separate content from much of the engine. New items, missions, events, and abilities are easier to author when they fit a stable vocabulary. Start with a few verbs that interact, such as move, spend, reveal, protect, damage, repair, and deliver.

The practical limit matters: a prototype with six well-tested interactions is more useful than a hundred entries whose only difference is a larger number. Every new rule increases the number of combinations that can go wrong. Record both intended interactions and prohibited ones.

8. Distribution and community are part of the product

Free access, easy launching, translations, useful tools, mod support, and visible maintenance can help a game find and retain an audience. The sampled projects expose several of these mechanisms. This is evidence that those channels exist, not proof of the share of growth caused by each channel.

For a first release, prioritize a working link, a readable title screen, controls that match the device, a save that survives reload, and a short shareable result. Community editors and multiplayer become sensible investments after people value the core game.

9. Copy the decision structure, then change the premise and consequences

An original adaptation should answer three questions: What familiar decision pattern is being reused? What new constraint changes that decision? What expressive work will be newly made? Switching colors and names on the same progression is a weak starting point.

For example, apply OpenTTD's capacity decisions to an emergency dispatch map where success is coverage before a deadline rather than profit. Apply Unciv's competing investments to a compact season where fatigue and an opponent's tendency change the value of each investment. Use original names, writing, art, audio, and balance data.

Engineering guidance for Claude

These are proposed implementation choices for new small games. They are not claims that every reference game uses the same architecture.

Default delivery target

Build a browser game with TypeScript and Vite. Use ordinary HTML controls for menus and readable management screens. Use Canvas for a moving board or action scene when it materially helps. Vite provides a TypeScript starter and produces static assets; confirm the actual installed runtime and dependency requirements before scaffolding. Source: Vite getting started.

Make the first release usable on a laptop and a narrow phone screen where the genre permits. Touch controls should be designed, not left as a side effect of desktop buttons. A rhythm game should declare its tested input and audio environment instead of implying all devices have equivalent latency.

Keep the first version self-contained after loading. A runtime LLM API, account system, purchase flow, or multiplayer service is outside these initial briefs. Claude can author and revise content during development without making every player's session depend on a model call.

Organize around stable responsibilities

Module	Responsibility	What should not live here
Simulation	Apply a command to game state, resolve rules, emit events	DOM access, audio playback, network requests
Randomness	Seeded generation and saved random state	Ad hoc random calls scattered through rules
Content	Validated definitions for teams, units, items, missions, maps	UI rendering or save migration code
Presentation	Read state, display choices, animate emitted events	A second version of the game rules
Persistence	Validate, migrate, save, restore, export, recover	Silent replacement of an unreadable save
Explanation	Turn recorded events into a concise result or recap	Invented reasons that the simulation did not record

For a turn-based game, a useful boundary is `applyCommand(state, command, rng) -> { state, events }`. In practice the random state must be part of the serialized session or passed and returned explicitly. Record a rules version so that a future balance patch does not silently reinterpret an old replay. For policy comparisons, generate and save external events separately from the player's decisions; using the same initial seed alone is insufficient if different choices consume different numbers of random draws.

For a real-time game, use a bounded simulation step and decouple it from drawing. Pause or explicitly handle tab suspension. For a rhythm game, derive judgments from the audio timeline plus a measured input offset, not from accumulated animation frames. MDN describes `AudioContext.currentTime` as an audio timeline that advances while the context runs; browser and hardware timing still require calibration and device testing. Source: MDN audio timeline.

Save and recovery requirements

Save enough to resume the exact decision boundary: schema version, rules version, seed and random state, player choices, world state, and the current phase. Validate a new snapshot before replacing the last known good one. Handle unavailable storage and quota errors visibly. Offer export/import and a clear reset action. Do not erase a bad save without giving the player a recovery option.

For compact prototypes, versioned local storage can be sufficient. Choose IndexedDB when save size, history, or structured persistence warrants it; do not add a database just to make the project look substantial. On restoration, prevent duplicated rewards and repeated end-of-turn processing.

Tests that protect the experience

Use targeted tests for rules and recovery, then test the actual interface. Appropriate checks include identical seed plus identical commands producing identical outcomes; resource conservation; valid phase transitions; no double reward after reload; content references resolving; generated maps containing a legal path; and every intended terminal outcome returning to a usable menu. Challenge games need a full win and a full loss; a narrative or teaching game needs its intended endings or comparisons verified.

A passing test suite does not establish that a game is fun. Include an observed play session, a description of the first confusing moment, and one change based on the observation. Report automated checks, manual checks, and untested assumptions separately.

Keep explanation honest

The result screen should describe the actual chain of events. "Your reserve was empty when the west line failed" is defensible if the event log shows both events. "Resting the captain would have guaranteed victory" is not defensible unless the game explicitly evaluated that scenario under appropriate assumptions.

For a sports simulation, show relevant strengths, costs, tendencies, and uncertainty. Do not promise real-world predictive accuracy. A fictional game can be transparent and strategically rich without pretending to be a validated sports forecasting model.

Six original build briefs

Brief A: One More Season

Priority: Best fit for Kevin. **Primary reference:** Unciv's competing investments. **Supporting lessons:** Shattered Pixel Dungeon's run identity and Lichess's useful review. **Complexity:** Moderate, with simulation balance as the main risk.

Premise. Run a fictional five-a-side, football-inspired club through a six-fixture season. Each fixture asks you to balance lineup quality, fatigue, training, and one opponent tendency. The desired feeling is that you recognize your team, make a difficult call, watch it matter, and immediately want another season. These are simplified original rules rather than a simulation of a real league's rulebook.

The loop. Read a short scouting report; choose five starters from a fixed eight-player roster; spend up to two preparation points; choose one of three tactical plans; resolve eight alternating possessions per team; review three decisive events; make one bounded development choice. Repeat for six fixtures and receive a season outcome.

Meaningful tradeoff. A tired star may still produce the best immediate offense, but using a weaker reserve lets the star recover for the next fixture. Training raises an attribute at a fatigue cost; rest improves current readiness. Aggressive tactics trade possession reliability for upside. Opponent tendencies are visible enough to support planning; they do not turn the match into a certain answer.

First playable scope. One fictional league containing the player team and six distinct scheduled opponents; eight fixed players on the player's roster; four attributes per player; three tactical plans; two preparation points before each match; twelve authored recap templates; six fixtures. The templates describe actual logged events and do not introduce additional random events or progression systems. A run succeeds with at least four wins and always reaches a season recap. Four wins is an initial tuning hypothesis, not an evidence-based difficulty promise.

Initial rules contract. Each team receives eight possessions, alternating between teams, and each possession scores 0, 3, or 7 points. The higher total wins; a tied match counts as a draw and contributes zero wins toward the season target. There are no on-field positions, substitutions during a match, overtime, or extra-point decisions in this first version.

Player attributes are Attack, Defense, Control, and Stamina, each an integer from 1 to 10. Fatigue ranges from 0 to 100. For match calculations, multiply Attack, Defense, and Control by `1 - fatigue / 200`; Stamina reduces fatigue gained from playing rather than acting as another offensive bonus. Each fixture grants two preparation points that do not carry over. One point either rests one player, reducing fatigue by 24, or trains one attribute by 1 up to 10 while adding 8 fatigue. Each player receives at most one preparation action per fixture. After the match, starters gain `24 - Stamina` fatigue and reserves lose 18; clamp fatigue to its valid range. After fixtures one through five, choose one player and increase one attribute by 1, at no fatigue cost, exactly once. There is no separate XP, recruitment, or equipment system.

At equal effective strength before matchup adjustments, use these starting probability distributions for scores of 0, 3, and 7: Patient = 25%, 55%, 20%; Balanced = 35%, 35%, 30%; Aggressive = 50%, 10%, 40%. Team attributes and a visible opponent tendency may shift up to 15 percentage points between the 0-point and 7-point outcomes. Keep the total at 100%, document the exact modifier formula, and record it with each possession. These numbers are transparent starting values to tune through the seeded comparison, not claims about real football.

First two minutes. Give the player a prebuilt roster and a visible problem: the strongest attacker is tired, while the first opponent is vulnerable to a patient style. Allow a lineup change and one training-versus-rest choice, then resolve the first possession promptly. Explain attributes in context.

Model and presentation. A pure match simulator emits possession events containing the chosen plan, fatigue influence, matchup terms, random roll, and outcome. Show a compact field or possession timeline rather than requiring a full action football engine. Teams, players, tactics, and recap templates live in validated data files. At commitment, resolve the match once and persist its complete result, event log, and random state before presenting playback. Reload resumes that stored result. Record separate completion flags for fatigue updates and the post-match development choice so neither can be applied twice.

Acceptance criteria. Each of two contrasting strategies reaches the four-win success target on documented seeds; compare them on the same small fixed seed set and report their different tradeoffs without claiming broad balance from a few runs. Fatigue must change an actual decision; every reported

decisive event must point to recorded data; reload must preserve the current phase; rerunning the same seed and decisions must reproduce the result. Verify both a successful season with at least four wins and an unsuccessful season with three or fewer, each reaching its recap. Observe whether players can identify one decision they would change after a loss.

Scope cap. No real teams, likenesses, recruiting database, transfer market, online league, fully simulated 22-player field, or multi-decade career. Later expansion should add one meaningful system after the six-fixture run is enjoyable.

Claude instruction. Implement Brief A as a separate, complete game. Start with one fixture and two contrasting viable plans, then expand to six fixtures. Preserve the ability to inspect the match math. Deliver a playable build, a short rules explanation, a seeded balance harness, save/recovery checks, and a sample recap produced from real simulated events.

Brief B: Blackout Dispatch

Priority: Best broad-audience experiment. **Primary reference:** OpenTTD's visible network bottlenecks. **Supporting lesson:** Mindustry's infrastructure under pressure. **Complexity:** Moderate; puzzle quality matters more than map size.

Premise. Keep five city districts powered during a twelve-turn storm. You place or reroute links on a small grid while demand and failures approach. The board should make it obvious why a district is lit, short of power, or disconnected.

The loop. Inspect the next turn's demand and a two-turn storm forecast; spend two actions placing, repairing, upgrading, or rerouting a link; commit the turn; watch supply move; inspect unmet demand and remaining reserve. Each decision competes for the same limited action and material budget.

Meaningful tradeoff. An efficient single trunk serves more demand cheaply but has a critical failure point. A redundant path costs actions and materials but survives a forecast outage. Players choose whom to protect when full coverage is temporarily impossible.

First playable scope. An eight-by-eight board, one generator, five districts, three link types, twelve turns, three handcrafted scenarios, and one seeded variation mode. Generator output and line capacity are explicit. Set each district's unmet-demand limit and the success conditions in the scenario data, and show them before play. Use visible turn resolution, not hidden real-time drains.

First two minutes. Connect a nearby district, watch it turn on, then reveal that a second district exceeds the shared link's capacity. Teach the difference between connectivity and throughput through a repairable decision.

Model and presentation. Use a directed graph with integer capacities and a deterministic allocation order shown to the player. Initial routing can be simple and inspectable; a technically sophisticated optimizer that silently makes the decisions would undermine the game. Render the board in Canvas or SVG with HTML controls and numeric labels. All information must remain understandable without relying on color alone.

Acceptance criteria. Energy is never created by a cycle; allocation rules match the displayed explanation; a replay resolves identically; at least two layouts solve the introductory map; failure identifies the disconnected or overloaded edge; an action can be previewed and undone before turn commitment.

Scope cap. No realistic electrical engineering, city economy, endless procedural map, online scoreboard, or hundreds of buildings. This is a logistics puzzle with fictional units, not an infrastructure planning tool.

Claude instruction. Build one polished twelve-turn scenario first. Make the supply path and the next threat visible. Demonstrate an efficient fragile solution and a robust expensive solution before adding variation. Deliver the scenario definitions, a deterministic flow resolver, readable failure explanations, and a complete win/loss/restart path.

Brief C: The Last Signal

Priority: Smallest narrative production footprint. **Primary reference:** A Dark Room's staged discovery. **Supporting lesson:** Bitburner's shift from manual work to automation. **Complexity:** Low to moderate; pacing and writing are the main risks.

Premise. Restore an abandoned relay station. At first, you power a receiver and decode a damaged message. Later, you decide which remote stations receive scarce power and which messages deserve your attention. The story changes the meaning of the machinery you have already learned.

The loop. Allocate power, decode a signal, choose a repair, unlock one automation rule, discover a new station, and decide whether to trust a request. Automation takes over understood chores and creates room for new decisions.

Meaningful tradeoff. Routing power to a distant sender slows repairs at home. Automating a low-priority task is efficient until a changing signal requires intervention. Avoid a fake moral choice with a clearly labeled good reward; let the consequences follow understandable resource commitments.

First playable scope. Three chapters, three resources, four stations, twelve authored message events, four automation rules, two endings, and a target first completion of roughly twenty minutes. The target is a design goal to test. Provide active ways to advance; do not pad the run with real-time waiting.

First two minutes. One working button, an incomplete signal, a choice of two repairs, and the first visible change to the station. Unlock the second resource only when the player has a reason to care about it.

Model and presentation. A data-driven state machine controls chapter gates. Each event defines prerequisites, costs, effects, and a one-time flag. Automations are a small safe rule vocabulary, not arbitrary JavaScript execution. Use an event log to prevent duplicate story rewards after reload.

Acceptance criteria. Every ending is reachable; each unlock introduces one meaningful new decision; no required action leaves the player stuck without feedback; recovery preserves story flags; players can explain what an automation does before using it; a session can be completed without idling.

Scope cap. No live AI-generated dialogue, infinite narrative, prestige layers, crafting encyclopedia, or daily login rewards. Finish a coherent small story before extending it.

Claude instruction. Create the first chapter with a satisfying reveal, then implement the entire three-chapter arc. Keep story content editable separately from state transitions. Deliver a chapter map, event schema, duplicate-reward protections, and both ending paths.

Brief D: Junkyard Courier

Priority: Best compact systems adventure. **Primary reference:** Endless Sky's jobs and loadout identity. **Supporting lesson:** Shattered Pixel Dungeon's resource decisions under uncertainty. **Complexity:** Moderate.

Premise. Deliver salvage across a flooded industrial archipelago in a tiny customizable boat. Your cargo, fuel, and equipment share limited space. Each contract creates a reason to choose a route rather than wander aimlessly.

The loop. Choose two contracts from three offers; load cargo and equipment; choose the next route node; resolve a visible hazard; decide whether to press on, repair, trade, or return; complete delivery and select one upgrade. A run ends after three deliveries or the loss of the boat.

Meaningful tradeoff. A bulky engine reduces travel exposure but occupies cargo space. A lucrative fragile shipment changes which route is safe. Repair supplies offer insurance but reduce the space available for profit. Show hazard categories and consequences before the irreversible route choice.

First playable scope. One node map, twelve locations, one boat, six modules, eight contract types, six hazards, and three deliveries per run. Use a turn-based route map and short encounter cards. Each generated map must contain a viable starter route.

First two minutes. Present one safe modest contract and one profitable bulky contract, show the storage conflict, and let the player launch. The first obstacle should reward reading the route rather than obscure item knowledge.

Model and presentation. A loadout validator checks space, compatibility, and resource costs. Contracts are state machines with explicit acceptance, delivery, failure, and reward conditions. Encounters are data plus small rule handlers. Save the map seed and resolved encounter state.

Acceptance criteria. Contracts cannot pay twice; cargo cannot exceed capacity; the starter equipment can finish at least one route; two loadouts lead to different sensible choices; a failed run identifies its resource or navigation mistake; saved runs resume without rerolling hazards.

Scope cap. No open-world real-time sailing, faction diplomacy, dozens of ship classes, procedural prose, or multiplayer. Add one module only when it changes a decision already present in the game.

Claude instruction. Build the contract-loadout-route loop first and finish one complete three-delivery run. Use original worldbuilding and placeholder shapes until the choices are enjoyable. Deliver map validation, cargo invariants, contract state tests, and at least two documented successful loadouts.

Brief E: Beatline Salvage

Priority: Best mastery-focused alternative. **Primary reference:** osu!'s precision and performance feedback. **Supporting lesson:** 2048's immediate control clarity. **Complexity:** Moderate to high because timing quality is the product.

Premise. Operate a two-lane salvage machine by timing left and right actions to an original beat. Accurate timing routes useful scrap; deliberate route switches create small scoring opportunities. The visual world should respond to the same rhythm the player is performing.

The loop. Play a ninety-second authored chart, receive timing feedback, inspect the section that caused errors, practice it, and retry. New challenges alter a limited vocabulary rather than introducing a new controller every song.

Meaningful tradeoff. A clearly signaled optional route offers more points with a harder pattern. Ordinary accuracy should remain rewarding. Difficulty selection changes density and spacing; it should not require buying a numerical advantage.

First playable scope. Two inputs, one original or properly licensed ninety-second music piece, three authored difficulty charts, one calibration tool, one practice loop, and a results screen with timing-error distribution. Use keyboard first, with touch support only after separate testing. Do not claim a mobile release is ready because two touch buttons exist.

First two minutes. Allow a short calibration and immediate play. Show early/late feedback and explain a miss without burying the player in statistics. Offer a retry with one action.

Model and presentation. Chart notes contain timestamps, lanes, and judgments. Use the audio clock as the reference and separate judgments from animation. Define pause, audio suspension, focus loss, and restart behavior. Keep chart data independent of audio rights and source code licensing.

Acceptance criteria. Simulated timestamps at judgment boundaries score correctly; restart resets clock offsets; pause/resume does not manufacture misses; the same recorded input sequence produces the same scoring under the same settings; a human device test checks perceived sync.

Scope cap. No commercial song imports, online rankings, multiplayer, beatmap marketplace, or automated difficulty-rating system. One convincing song is the release gate.

Claude instruction. Prove timing correctness and perceived sync before adding visual effects. Deliver one complete song, calibration, practice mode, timestamp-based scoring tests, and a candid list of tested devices and remaining latency limitations.

Brief F: Roommate Republic

Priority: Best fit for a classroom-adjacent experiment. **Primary reference:** The Evolution of Trust's prediction-experiment-explanation cycle. **Supporting lesson:** Lichess's review and replay. **Complexity:** Low to moderate.

Premise. Four fictional roommates share a household budget and recurring chores. You choose a rule for splitting costs, predict what it will encourage, and watch a short simulation. Change one rule and replay the same conditions to discover its consequences.

The loop. Inspect a household problem; predict which policy works best; choose a policy; run four simulated weeks; compare the outcome with the prediction; adjust one assumption; explain the difference. The goal is insight and experimentation rather than a permanent leaderboard.

Meaningful tradeoff. Equal contributions are easy to understand but interact differently with unequal incomes. Usage-based billing can reduce waste while creating incentives to underreport. A shared reserve protects the household but restricts spending today. All assumptions should be visible and editable.

First playable scope. Four agent behavior models, three scenarios, four policy choices, four weeks per scenario, one reproducible surprise expense, and one comparison view. Use fictional units and situations. This is a transparent model of incentives, not advice about what a specific household should do.

First two minutes. Ask the player to predict who pays and who changes behavior under one simple rule, then run the scenario. Put the explanation after the player's prediction so it answers an experienced question.

Model and presentation. Keep the budget ledger, agent behavior, scenario assumptions, and explanation text separate. Each expense and transfer has an origin and destination. Generate the external event schedule once and save it; a policy replay reads that same schedule rather than drawing a fresh sequence. Explain which conclusions depend on simplified behavior assumptions.

Acceptance criteria. Money is conserved except for explicitly modeled income and expenses; changing a policy changes only intended rules; the comparison uses the same event seed; every explanation can be traced to the ledger; the game never treats an invented behavioral assumption as a fact about real people.

Scope cap. No bank accounts, student data, real investment returns, financial recommendations, or internet multiplayer. Use a teacher-view debrief only after the player experience works.

Claude instruction. Build one scenario that produces a surprising but explainable result, then add the other two. Deliver a readable assumptions panel, a transaction ledger, a same-events comparison, and three short debrief prompts that ask players to identify evidence and limitations.

A practical build sequence

Stage	Deliverable	Gate before expanding
1. Core decision	One playable situation with feedback	A player can describe the choice and its cost
2. Complete run	Beginning, escalation, win/loss, restart	No dead end; at least two sensible approaches
3. Persistence and explanation	Resume, recovery, recap	A reload preserves state and the recap matches events
4. First observation	Five new players plus optional genre enthusiasts	Record confusion and voluntary replay; revise the loop
5. Content and presentation	A small coherent set of scenarios	Added content changes decisions and remains understandable
6. Release candidate	Tested static build and documented limitations	Full run verified on the claimed devices

These are completion gates, not time estimates. A large existing open-source project often represents many years of contributors' work. Its current feature list is not a responsible starting scope for one coding session.

Playtest protocol and stop rules

Invite five people who have not seen the design. Let each play without coaching. Record time to the first meaningful action, the first confused question, whether the player can state the goal, the decision they regretted, and whether they voluntarily begin another run when given a neutral choice to stop. Ask genre enthusiasts separately; do not mix the groups into a broad market claim.

Suggested early gates are hypotheses: at least four of five can explain the goal and one tradeoff after their first run, and at least three of five voluntarily retry. A five-person observation can expose obvious problems but cannot establish retention, market demand, or a reliable percentage of future customers. Keep the qualitative notes.

If players do not understand why they lost, improve feedback. If they understand and do not want another attempt, change the decision structure or pacing. If everyone follows one obvious best strategy, rebalance or remove the fake choices. If adding content repeatedly breaks saves or core rules, fix the boundaries before expanding. Do not answer each failure by adding more upgrades.

Copy/paste master instruction for Claude

You are the game designer and implementing engineer for one original small game. Read this report as a collection of researched examples and proposed design hypotheses. Its source games are references; do not reproduce their names, characters, music, maps, writing, distinctive artwork, or source code without an explicit reuse decision and a verified compatible license.

Default selection: Brief A, One More Season. If Kevin names another brief, use that one. Build exactly one game. Start in a new project unless Kevin explicitly identifies an existing repository to extend. If an existing repository is specified, inspect its instructions, current architecture, build commands, tests, and uncommitted work before changing it.

Before implementation, write a short design contract: the player fantasy, the repeated decision, the scarce resource, the loss condition, the target session, the two plausible strategies, the first sixty seconds, and the features deliberately outside the initial build. Make routine decisions yourself and state assumptions. Ask only if a missing answer materially changes the scope or authorization.

Implement the smallest complete version. Use the selected brief's concrete counts as scope caps. Prioritize a responsive action, clear feedback, a consequential tradeoff, a complete run, and immediate restart. Add content only after those are working. Preserve a clear route from the title screen to the actual game. Every visible button must perform its stated action or clearly explain an unavailable action.

Prefer TypeScript and a static browser build for these briefs, adapting to an existing repository when appropriate. Separate simulation, presentation, content definitions, randomness, persistence, and explanations. Use a seed and versioned state. A turn-based game should have a testable command boundary. A rhythm or action game must use the correct time source and explicit pause/focus behavior.

Create original content. Use simple original placeholder graphics and audio where needed; keep an asset manifest recording creator, source, license, and modifications. Do not infer that a repository's code license covers its music or artwork. Do not add a runtime LLM service, account system, purchases, online multiplayer, or deployment unless the task specifically calls for it.

Implement recovery before expanding content. Reload must preserve the current decision boundary without awarding anything twice. Validate saves and retain a recovery path. Provide clear export/import or backup behavior where appropriate. Never silently destroy a player's only save.

Verify the rules with meaningful tests: determinism where applicable, resource or score invariants, legal transitions, content validation, save restoration, and every intended terminal outcome. Challenge games must include a full winning and losing run; narrative and teaching games must cover their intended endings or comparisons. Verify the actual interface on the devices you claim to support. Report which checks you ran, their results, and any remaining limitations. Do not claim that passing tests proves the game is fun.

Provide a short playtest script and a visible debug mode that helps inspect decisions without changing normal play. Recaps must use recorded events. Any counterfactual or predicted benefit must be labeled and evaluated honestly. In the sports brief, do not invent realistic accuracy for the simulation.

Finish with the runnable project, exact local run/build commands, the production build, a concise design explanation, an asset/license manifest, a verification summary, and the single most useful next experiment. Do not stop after scaffolding or a planning document if a coding environment is available. If you cannot execute code in the current environment, say so and provide a complete bounded implementation handoff without pretending the game has been built or tested.

Evidence, reuse, and next decision

What is established and what remains a hypothesis

Question	Evidence level in this report	Practical meaning
Does the repository exist, and what did its metadata show?	Direct GitHub reads on the research date	Use the dated snapshot; refresh before making a new ranking.
What does the inspected code implement?	Targeted source reads and official documentation	Follow the linked files; the entire codebase was not audited.
Is there a free playable route?	Project documentation or an official distribution page	Read the platform and asset qualifications in each case.
Why might the game be engaging?	Design analysis grounded in observed mechanics	Treat the explanation as a testable hypothesis.
Which feature caused popularity?	Not established	No controlled attribution or comparable retention dataset was available.
Will a proposed new game succeed?	Not established	Build the smallest full run and observe actual players.

The observed projects have different ages, audiences, team sizes, distribution channels, and relationships to existing franchises. Those differences prevent a clean causal comparison. Long-lived projects also benefit from years of accumulated attention. A high star count is a reason to investigate a project, not proof that each feature is worth reproducing.

Reuse boundary

The recommended briefs use original implementations and newly authored content. Direct reuse is a separate decision. Read the actual code license, retain required notices, and inspect each asset's origin and terms. A repository's automatic license badge is a starting clue. It is especially incomplete when a game downloads external assets, includes music under separate terms, or mixes code and artwork licenses.

Specific examples found in this review include Bitburner's Apache 2.0 plus Commons Clause terms, the separately licensed sounds credited by The Evolution of Trust, the external original-game assets used by OpenRA's supplied mods, the asset-by-asset copyright inventory in Endless Sky, and the distinction between osu!'s code and its branding or beatmap content. These distinctions affect what can be redistributed; they do not prevent learning from the design.

Source reading plan for a future implementation

Claude should start with the files cited in the relevant case rather than attempting to absorb an entire mature repository. Read the selected rule or progression module, its data definitions, and one consumer that turns its output into feedback. Write down the transferable boundary in plain language. Then implement the new game's equivalent from its own design contract.

If the source changes after this report, use a pinned commit where provided or record the new commit before drawing implementation conclusions. Repository paths can move. Release tags, branch tips, latest pushes, and current published builds are different things; do not assume one proves the others are identical.

The next concrete step

Use **Claude_Game_Build_Start_Here.md** to begin **One More Season**. The first reviewable milestone is one complete fixture with a real lineup tradeoff, recorded possession events, a readable recap, and immediate replay. Expand to the six-fixture season after the single-fixture decisions are understandable and worth repeating.

If the goal is a game for people with no sports interest, choose **Blackout Dispatch** instead and prove one twelve-turn scenario with two viable network layouts. Keep the same requirement: a complete playable outcome before a larger feature list.